Noc-Dashboard

System & User Guide

Issue	1
Date	2026-08-28
Prepared By	Noc-Dashboard Engineering





NOC-DASHBOARD

Contents

1 Introduction 4
1.1 About Noc-Dashboard 4
1.2 Who This Guide Is For 4
1.3 Document Conventions 4

2 System Architecture 5
2.1 Architecture Overview 5
2.2 Why Two Independent Site-Status Sources 5

3 Core Features 6
3.1 Dashboard 6
3.2 Map View 6
3.3 Partner Contact 6
3.4 Reports 7
3.5 Alerting Overview 7

4 Interface & Navigation Flow 7
4.1 Navigation Map 7

5 Notifications & Alerting 8
5.1 Telegram Alerts 8
5.2 Teams Webhook Alerts 9

6 Technology Stack 9
6.1 Backend 9
6.2 Frontend 10
6.3 Export & Alerting 10

7 Clear-and-Clean Initiative 10
7.1 Map Data Accuracy 11
7.2 Codebase Health 11

8 User Guide 11
8.1 Reading the Dashboard 11
8.2 Using the Map 12
8.3 Managing Partner Contacts 12
8.4 Filtering & Exporting Data 12

9 Support & Reference 12
9.1 Troubleshooting Discrepancies 12
9.2 Document Status 13

1 Introduction

This chapter introduces Noc-Dashboard, who this guide is for, and the conventions used throughout it.

1.1 About Noc-Dashboard 4
1.2 Who This Guide Is For 4
1.3 Document Conventions 4

1.1 About Noc-Dashboard

Noc-Dashboard is an internal, browser-based monitoring dashboard that gives NOC staff a single, always-current view of network alarms, incidents, site status, and partner contacts, without needing to log into Streamline or any source system directly.

The application reads from two upstream systems — PostgreSQL (the Streamline platform's own database) and MSSQL (the NOC's internal inventory and alarm systems) — and keeps a local cache that background jobs refresh on independent schedules, so every page loads instantly and the upstream databases only ever see a small, steady stream of sync queries.

NOTE

Noc-Dashboard has no login system by design. It is a shared, read-mostly operations display. Every write action (acknowledging an alarm, editing a partner contact) still records who performed it, captured from a name field, not a session.

1.2 Who This Guide Is For

- NOC engineers and shift leads using the dashboard day to day.
- New team members who need a guided overview of what each page does.
- Anyone maintaining or extending the application going forward.

1.3 Document Conventions

The following conventions are used throughout this guide.

Symbol / Style	Meaning
NOTE	Supplementary information that makes a feature easier to use correctly.

Symbol / Style	Meaning
IMPORTANT	Something that will cause visibly wrong or confusing results if missed.
VERIFIED	A statement checked directly against the running application or its test suite, not a design intent.
Bold text	A literal label as it appears on screen, e.g. Add Partner.

2 System Architecture

This chapter describes how data flows from the two upstream systems into the pages a NOC engineer actually sees.

2.1 Architecture Overview 5
2.2 Why Two Independent Site-Status Sources 5

2.1 Architecture Overview

A layered pipeline keeps the dashboard fast and the upstream systems protected: sources feed schedulers, schedulers feed one local cache, and every page reads only from that cache.

Layer	Technology	Role
Data sources	PostgreSQL (Streamline) & MSSQL (NOC inventory)	System of record for sites, incidents, alarms, and partner data.
Background sync	Python scheduler threads	Pull each source on its own interval and normalize rows into a common shape.
Local cache	SQLite (instance/localstorage.db)	Single always-fresh copy every page reads from, never the live databases directly.
Web layer	Flask + Jinja templates	Dashboard, Map, Partner Contact, and Reports pages, plus CSV/XLSX export.
Alerting	Telegram bots & Teams webhooks	Independent background jobs reading the same cache to push proactive notifications.

2.2 Why Two Independent Site-Status Sources

Site and alarm data has historically lived in Streamline's PostgreSQL database. On-Air site status now comes from a second, independent source instead — MSSQL's own site and network-element tables — because it reflects real network inventory more directly and updates sooner than Streamline's own status field. Planned and Other site statuses remain sourced from PostgreSQL, since MSSQL has no equivalent concept for a site that isn't on air yet.

NOTE

The two sources are deliberately kept independent rather than merged. If MSSQL and PostgreSQL ever disagree about a specific site, the Map shows both states rather than silently picking one, so a real data discrepancy stays visible instead of being hidden by a tie-breaking rule.

3 Core Features

This chapter describes each of the five main areas of the application.

3.1 Dashboard 6
3.2 Map View 6
3.3 Partner Contact 6
3.4 Reports 7
3.5 Alerting Overview 7

3.1 Dashboard

Live badge counts and drilldown tables for ISP POP / EZE / TCT alarms, Incidents & INC-WOs, RAN Status (Main AC, Site Down, Tech Down, Voltage), VIP/VVIP sites, Mentor sites, DG On, and Alarm Site Down. Each badge opens its own filterable drilldown with a CSV export.

Three chart cards summarize trend data: Incident Age — Open (a donut chart bucketed by how long an incident has been open), Incidents Closed — Week over Week, and Incident Volume — Week over Week. The latter two drilldowns support an additional checkbox filter by Incident Category, alongside the existing day filter — see Section 8.4.

3.2 Map View

Every On-Air, Planned, and Other site is plotted with a real coordinate, alongside every ISP POP, colored by status and vendor. The Map supports free-text search, province / district / commune boundary drill-in, automatic background refresh, and a styled .xlsx export of whatever is currently shown.

3.3 Partner Contact

A directory of partners and their escalation contacts, backed directly by MSSQL. Supports creating a partner with its first contact, adding further contacts to an existing partner, editing, deleting, and a dedicated Manage Circuits page per partner.

3.4 Reports

A Custom Dashboard view combining several cards — INC-WO status, resolved work orders, and category-power alarms — for a single consolidated export.

3.5 Alerting Overview

Telegram bots post Site Down and INC-WO alerts as table images with a matching .xlsx attachment, on their own schedules. Teams webhooks post the same class of alerts to a channel. Both read the same local cache the pages do — see Chapter 5 for the full alerting flow.

4 Interface & Navigation Flow

This chapter shows how the main areas of the application connect, from entry to export.

4.1 Navigation Map 7

4.1 Navigation Map

There is no login step. Opening the dashboard's URL goes straight to live data.

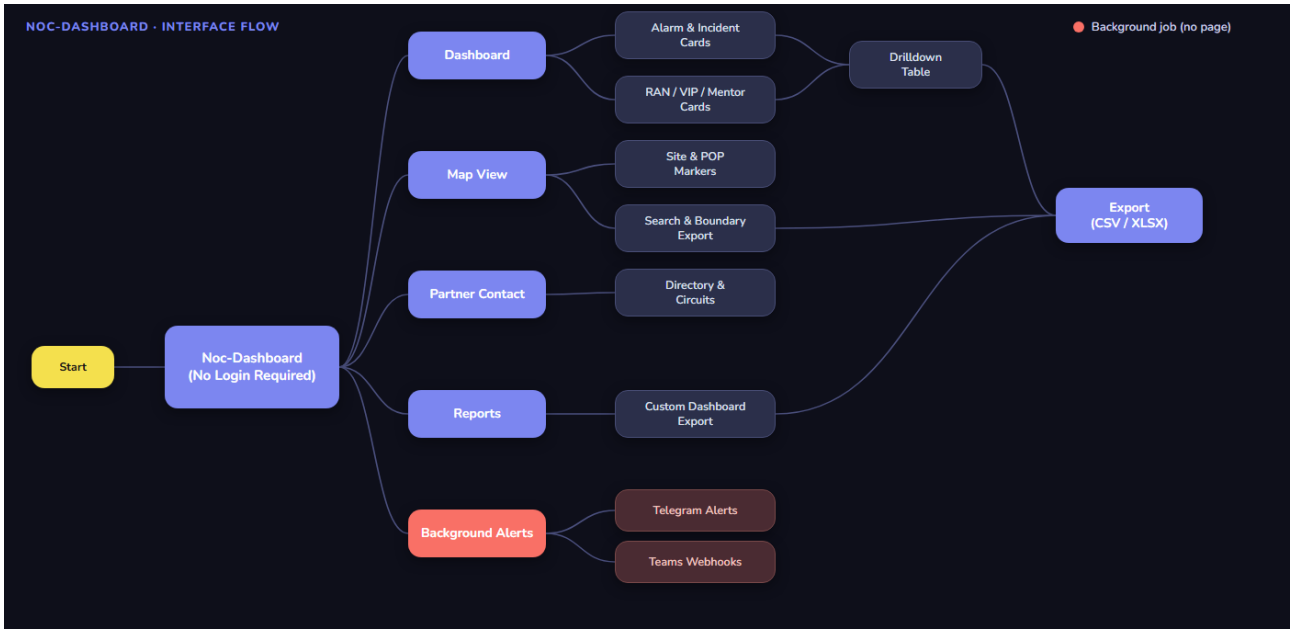


Figure 4-1 Noc-Dashboard's top-level navigation, from entry to the shared CSV/XLSX export every drilldown, Map search, and Report ends at.

NOTE

Background Alerts (shown in red in Figure 4-1) is not a page a user navigates to. It runs continuously on the server, independent of anyone having the dashboard open, and is included to show it reads the same cache as every visible page.

5 Notifications & Alerting

This chapter describes the background alert jobs — what each one watches, where it sends, and on what schedule. Bot tokens and chat/group IDs are configured in the application's environment file and are not reproduced here; the group names below are illustrative labels for the role each destination plays.

5.1 Telegram Alerts 8
5.2 Teams Webhook Alerts 9

5.1 Telegram Alerts

Four independent Telegram bots run on their own schedules, each reading the local cache and posting to its own configured chat. Every send also mirrors its rows into a matching cache table, so the last alert sent is always inspectable independently of the dashboard's own cards.

Alert	Watches	Illustrative Destination	Default Schedule
Site Down	Incident Site Down, past a configurable outage threshold	NOC – Site Down Alerts	Every 4 hours
INC-WO Site Down	Open INC-WO Site Down rows, L2 On-Site only	NOC – INC-WO Site Down	Every 15 minutes
INC-WO Category Power	Open INC-WO Power rows, L2 On-Site only	NOC – INC-WO Power	Every 15 minutes
INC-WO General	Every other open INC-WO category, L2 On-Site only	NOC – INC-WO General	Every 15 minutes

NOTE

Each alert is sent as a table image (for readability in Telegram) with a styled .xlsx attachment carrying the same rows, and falls back to a plain-text list if image rendering isn't available.

5.2 Teams Webhook Alerts

A separate scheduler posts the same class of active-alarm summaries to a Microsoft Teams channel via incoming webhook, on its own interval. Each card's webhook URL is configured independently, so an individual card can be silenced without affecting the others.

6 Technology Stack

This chapter lists the technologies Noc-Dashboard is built on.

6.1 Backend 9
6.2 Frontend 10
6.3 Export & Alerting 10

6.1 Backend

Component	Technology
Web framework	Flask
Templates	Jinja2

Component	Technology
Postgres access	SQLAlchemy + psycopg2
MSSQL access	pymssql
Local cache	SQLite (WAL mode)
Production server	Waitress

6.2 Frontend

Component	Technology
Map rendering	Leaflet
Icons	Tabler Icons
Typography	Nunito, Poppins, Exo2 (self-hosted)
Interactivity	Vanilla JavaScript, no frontend framework

6.3 Export & Alerting

Component	Technology
Spreadsheet export	openpyxl
Alert table images	Pillow
Telegram delivery	Telegram Bot API (HTTP)
Teams delivery	Incoming webhook (HTTP)

7 Clear-and-Clean Initiative

This chapter summarizes a focused pass to keep the Map's data accurate and the codebase easy to maintain.

7.1 Map Data Accuracy 11
7.2 Codebase Health 11

7.1 Map Data Accuracy

- On-Air sites now come exclusively from MSSQL's own site and network-element inventory — no blending with Postgres, no guessed fallback for a site MSSQL doesn't know about.
- Planned and Other sites remain exclusively from PostgreSQL, matching Streamline's own counts exactly.
- Vendor (NMS) for On-Air sites now reads directly from live network-element inventory instead of an indirect, partially-confirmed code mapping.
- A site the two sources disagree on is shown once per source, so the disagreement stays visible rather than being silently resolved.

7.2 Codebase Health

- Removed a dead, unused constructor parameter (and its import) carried through more than a dozen test call sites for no functional reason.
- Reviewed and condensed comments and docstrings across the application to short, professional descriptions that keep the important technical facts and drop restated-the-obvious prose.
- Removed stray, out-of-place files unrelated to the application.

NOTE

Every change in this initiative was verified against the full automated test suite and a static unused-code check (pyflakes) after each step, with zero functional changes — only what the data shows and how the code reads.

8 User Guide

This chapter is a practical walkthrough of the most common tasks.

8.1 Reading the Dashboard 11
8.2 Using the Map 12
8.3 Managing Partner Contacts 12
8.4 Filtering & Exporting Data 12

8.1 Reading the Dashboard

Each card's badge shows a live count. Click any badge to open its drilldown table.

8.2 Using the Map

- Use the status filter (top left) to switch between On Air, Planned, Other, and Site Down, or a specific vendor under On Air.
- Search by site code, POP code, or village name using the search box.
- Draw a boundary by searching a province, district, or commune to see a scoped list and export.
- The map refreshes itself automatically — no manual reload needed for status or Site Down changes on sites already shown.

8.3 Managing Partner Contacts

- Use Add Partner to create a new partner with its first contact in one step.
- Use the row's Add Contact button to attach another contact to an existing partner — this never creates a duplicate partner record.
- Use Manage Circuits on a partner's row to add, edit, or remove its circuits.

8.4 Filtering & Exporting Data

Every drilldown table, the Map, and Reports each have their own export control — CSV for tabular drilldowns, styled .xlsx for the Map and Reports.

The Incidents Closed and Incident Volume drilldowns additionally support a category filter: open the Filter by Category control, check one or more categories (only the categories actually present in that table are listed — a table containing only Site Down and Customer Complaint rows shows only those two), and select Apply. Checking more than one category shows rows matching any of them; the export respects the same selection.

9 Support & Reference

This chapter covers where to go for help or further detail.

9.1 Troubleshooting Discrepancies 12
9.2 Document Status 13

9.1 Troubleshooting Discrepancies

For questions about a specific card's logic, data source, or an unexpected count, check the relevant drilldown's date filter and export first. Most apparent discrepancies trace back to a sync-timing window (every background job refreshes every few minutes, not instantly) rather than a data error.

9.2 Document Status

NOTE

This document describes the system as of the date on the cover page. Feature details — especially sync intervals, badge names, and vendor-mapping rules — may change; treat the running application as the source of truth over this document if the two ever disagree.